BLM1031 Structural Programming - Project

SUBMISSION INFORMATION

The project must be submitted to the project module defined in the online.yildiz system by **Sunday, June 7, 2026, at 23:59**, according to the rules below.

Submission format:

StudentID.zip
(Inside the ZIP file, there must be only StudentID.c source code file and StudentID.pdf report.)

Uploading project links from cloud storage systems such as Google Drive, OneDrive, etc. to the online.yildiz.edu.tr system will **strictly not be accepted**. Only direct file uploads are allowed.

Late submissions cannot be made after the system closes. In addition, submission via email or any other method is not possible. Please keep a screenshot showing that your submission to the system was completed successfully as proof. Objections and excuses without proof will not be considered.

IMPORTANT RULES FOR PROGRAM AND REPORT DESIGN

First, read the general coding rules in the document below:

https://docs.google.com/document/d/1ySbmyym0s3VGGVW_Zm5vN7o7N0ttxdgPZ4yorHt5z7E/edit?usp=sharing

In addition to these rules, please pay special attention to the following:

- **Each operation must follow a modular design approach.** In other words, solutions containing only the main function or repetitive code performing the same task will be considered invalid.
- **The project must be coded in ANSI C.** Solutions written in other languages, including C++, will be considered invalid.
- **Compile your code using DevC++ or CodeBlocks before submission.** If the code produced using other IDEs or compilers does not run properly on our side, the submitted solution will be considered invalid.
- Pay attention to **variable and function naming, indentation, and writing clear and understandable code**. Before each function definition, include a comment explaining the function's inputs and outputs.
- **Do not use global variables, static variables,** continue, **or** goto **statements.** The break statement may only be used within switch operations. Solutions violating this rule will be considered invalid.
- Your report must include:

- A cover page and your contact information
- Step-by-step screenshots of every working part of the program
- An analysis of the code you designed for pathfinding in both manual and automatic modes using the sample input given in Figure-1
- A detailed explanation of the data structures (struct) designed for the game and user performance

- When including screenshots in the report, show only the relevant section of the screen. Including unclear or full-screen screenshots may result in point deductions.

PROJECT TOPIC:

A number-matching game will be developed on an N×N matrix. For example, for N=5, assume that the numbers between 1 and 5 are placed in pairs on the matrix shown in Figure-1 (colors are not important). Using the algorithms you develop, you are required to find paths that connect the matching numbers on the matrix according to the rules specified in the details section below.

Figure 1 Game Matrix

PROJECT DETAILS:

1. **Matrix Creation Rules:** The number placement game on the matrix must start by receiving the matrix size N from the user. According to this N value, numbers must be placed on the matrix using two different modes. You may use 0 or -1 as the value for empty cells.

   a. **Random Placement Mode:** According to the entered N value, each number between 1 and N must be placed randomly on the matrix exactly twice.

   b. **File Reading Mode:**The numbers provided in the additional files must be placed onto the matrix by using file reading functions.

   Note: Some example inputs given in item 1b are also provided together with their solved/matched versions.

2. **Matching Rules:** The numbers must be connected according to the following conditions:

a. Identical numbers must be matched with each other.

b. N different non-intersecting paths must be obtained. (Example: the 5 paths shown in Figure-2)

c. There must be no empty cells left in order to complete the game.

When the game is completed, a result similar to Figure-2 should be obtained.

2	2	2	2	5
2	1	5	5	5
2	1	5	4	3
2	1	5	4	3
4	4	4	4	3

Figure 2 Completed Game Matrix

3. **Rules for Making Moves During Matching:** In addition to the rules given in Section 2, the matching operations must be implemented in two different ways.

   a. **Automatic Matching Mode:** The algorithm you develop must automatically find the paths according to the rules specified in Sections 1 and 2. During this process, the generated paths and operations such as undoing incorrect moves — similar to the manual mode explained below — must be reported on the screen. Finally, when the game is completed, the matrix showing the resulting paths must be printed on the screen similar to Figure-2.

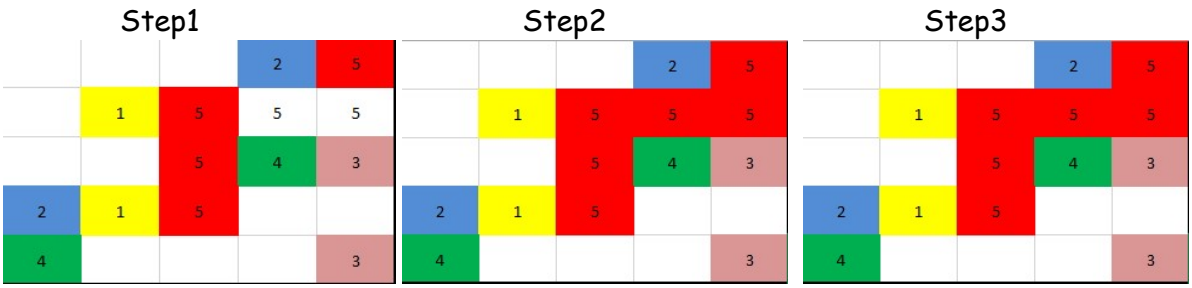
   b. **Manual Matching Mode:** The matching operations in this mode must work as shown in the example below. In this mode, paths must be created by filling the cells between two given matrix indices according to the 4-neighborhood rule (right, left, up, down).

      For example, in the fragment below, the process of matching two different cells containing the number 5 is demonstrated step by step during gameplay.

      Step 1: Source: (3,2), Destination: (1,2)

      Step 2: Source: (1,2), Destination: (1,4)

      Step 3: Source: (1,4), Destination: (0,4) Numbers are matched



NOTE: If an incorrect move is made in either the manual or automatic game mode, the move must be undoable (UNDO), and the game must never terminate because of a wrong move. As shown in the previous example, when a matching operation is undone, the matrix must return to its previous state. An example case is given below.

4. **Game Playing Rules:** According to the rules given in the first 3 sections, the game must be playable by M users. Each player should be able to play as many games as they want, and the score of each game must be recorded at the end of the game.

5. **Program Interface:** The game to be played using the designed algorithms must be implemented according to the following rules. Additional features may also be added.

a. Main Menu

i. Create Random Matrix
ii. Create Matrix from File
iii. Show User Scores
iv. Exit

b. Game Menu : When option (i) or (ii) is selected from the main menu, the username must first be requested, and the game should continue until the user chooses to return to the Main Menu from one of the following options:

i. Play in Manual Mode
ii. Play in Automatic Mode
iii. Return to Main Menu / Exit

6. User Scores

Develop a formula to calculate the score obtained by each user after every game and save the user scores according to this formula. The scoring formula must include the following parameters:

- Game completion time
- Whether the game was played in manual or automatic mode
- Matrix size
- Whether the matrix was generated randomly or loaded from a file
- Number of incorrect moves (UNDO count)
- Number of games played by the user

7. Saving Game Moves :

Design an appropriate data structure to save the game moves using an move/redo mechanism.This data structure must also be stored in a file so that even if the game is closed and reopened, the game can continue from the saved state.

8. Player Performance Records

The performance information of M different players must be stored in a file. Users should be able to view the performance information of all players whenever they want.

9. Project Evaluation

The report and the code will each be graded separately out of 100 points.
The final project grade will be calculated using the following formula:
Project Grade=20% Report+80% Code

Code Evaluation

- Codes related to the Main Menu and Data Structure Design: **10 points**
- Coding the Manual Gameplay Module: **45 points**
- Coding the Automatic Gameplay Module: **25 points**
- Coding the Game State Management (Move / Undo): **10 points**
- Storing Player Performance Information: **10 points**